Choosing the Right Derivative: A Practical Comparison of Julia's Automatic Differentiation Strategies

Jason liu
Master Student Software Engineering
University of Antwerp
Antwerp, Belgium
jason.liu@student.uantwerpen.be

***Abstract*—Automatic differentiation (AD) is the core mechanism behind differentiable programming, enabling gradients to be computed through arbitrary programs. This paper compares four AD strategies in Julia, a self-implemented forward mode using dual numbers, a self-implemented reverse mode using a tape, Zygote.jl, and Enzyme.jl, across three dimensions: correctness, runtime performance, and implementation complexity. Experiments are conducted on a parameter fitting problem using the Lotka-Volterra predator-prey ODE system. Results show that Enzyme is the fastest by a large margin due to LLVM-level differentiation, forward mode is the simplest to implement, and Zygote suffers significant overhead on scalar simulation code due to heap allocation per operation.**

***Index Terms*—Julia, Automatic differentiation, AD, Forward, Reverse, Enzyme, Zygote**

I. Introduction

Differentiable programming is a paradigm in which programs are written such that derivatives can be computed through them automatically. This enables gradient-based optimization, the backbone of machine learning, scientific parameter estimation, and numerical simulation, to be applied to arbitrary code rather than only to hand-derived formulas.

Automatic differentiation (AD) is the computational technique that makes this possible. Unlike symbolic differentiation, which manipulates mathematical expressions, or numerical differentiation, which approximates derivatives by finite differences, AD computes exact derivatives by applying the chain rule systematically through every operation in a program. There are two fundamental modes: forward mode, which propagates derivatives alongside values in a single forward pass, and reverse mode, which records the computation and recovers all gradients in a single backward pass. [1]

In practice, multiple AD systems exist with different design philosophies and performance characteristics. This paper evaluates four strategies available in Julia and examines how they differ in correctness, speed, and the burden they place on the programmer.

II. Background

Automatic differentiation exploits the fact that any program is ultimately composed of primitive operations, each with a known derivative. By applying the chain rule at every step, the derivative of the whole program is built up incrementally [1].

Forward mode AD attaches a derivative component to every value in the computation, requiring n passes for a function with n inputs. Reverse mode AD records the computation during a forward pass and recovers all n gradients in a single backward sweep, making it the preferred choice when n is large and the output is a single scalar. [1], [2]

The Lotka-Volterra predator-prey model [3] serves as the experimental vehicle, a system of two coupled ODEs describing population dynamics of prey and predators. We frame it as a parameter fitting problem: given observed trajectories, find the parameters minimizing a least-squares loss, and use AD to compute the gradient of that loss.

III. Experiment & Results

Our experiment is divided into four sections. First, we verify that all differentiation techniques produce correct gradients by comparing them against finite differences (correctness). Next, we measure the time each technique takes to compute those gradients (timing). Then, we scale the problem by increasing the number of parameters to observe how each method behaves under a heavier workload.

The experiment is based on the Lotka-Volterra predator-prey model [3], a system of two coupled ordinary differential equations that describes the population dynamics of two interacting species, prey (rabbits) and predators (foxes). Rather than solving the equations directly, we frame the problem as parameter fitting: given observed population trajectories generated from known true parameters, we compute the gradient of a least-squares loss with respect to all model parameters. This gives us a realistic many-inputs-to-one-output problem, which is precisely the setting where the difference between forward and reverse mode automatic differentiation becomes most apparent.

A. Methods

Here we will describe the four methods that we choose to benchmark, where two of them are custom made.

a) *Forward mode (self implemented):*

Forward mode automatic differentiation is implemented using dual numbers. A dual number carries two components

simultaneously: the actual value of the computation and its derivative with respect to one input. By overloading all arithmetic operators to propagate both components according to the chain rule, any existing function can be differentiated without modification.

```
struct Dual
    value::Float64
    deriv::Float64
end
```

To compute the gradient of a function with n inputs, forward mode must perform n separate passes, one for each input parameter, seeding that input's derivative component to 1.0 and all others to 0.0. This is the fundamental cost of forward mode: the number of passes scales linearly with the number of inputs. [1], [4], [5]

Note: to keep the implementation even simpler, we only overloaded the basic things and some other things we needed. To make this fully functional, you need to overload way more than what we did.

b) *Reverse mode (self implemented):*

Phase 1 — Forward pass: You run the computation normally, but secretly record every single operation onto a "tape" as you go. You're not computing any derivatives yet, you're just logging what happened and saving the input values of each operation, because you'll need those values later.

Phase 2 — Backward pass: You seed the output with a gradient of 1.0 (meaning "the output is 100% sensitive to itself"), then walk the tape backwards. At each recorded operation you apply the local derivative rule and push the gradient back to the inputs of that operation. By the time you've walked the entire tape, every input has accumulated its gradient.The crucial difference from forward mode is that one backward pass gives you all gradients simultaneously. You don't re-run the simulation for each parameter, you run it once, record it, then read all gradients off in one backward sweep. [1], [6]

Note: again we overload only what is needed to keep the implementation at minimum.

c) *Zygote:*

Zygote.jl operates using source-to-source transformation, where it parses existing Julia code and generates new Julia code specifically designed to compute gradients, rather than building a computational graph or relying on operator overloading.

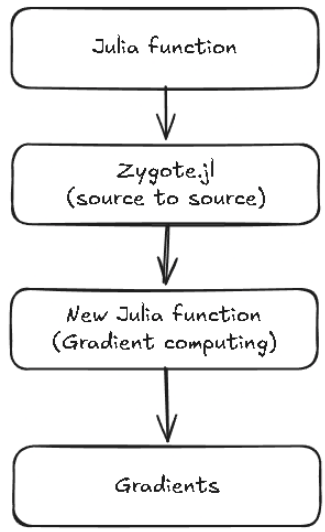

Fig. 1. Zygote workflow

At its core, Zygote uses reverse-mode automatic differentiation, which is ideal for deep learning scenarios with many input parameters and a single scalar output like a loss function. [7], [8]

d) *Enzyme:*

Enzyme.jl is a high-performance automatic differentiation (AD) system for Julia that operates by binding to the LLVM-based Enzyme differentiator, allowing it to compute derivatives of Julia functions by analyzing and transforming LLVM intermediate representation (IR) rather than source code. [9], [10]

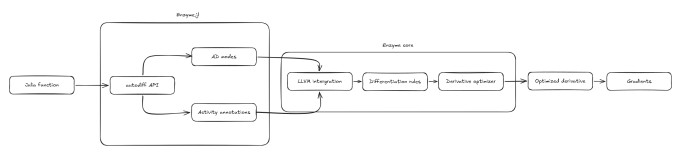

Fig. 2. Zygote workflow

Enzyme differs from Zygote, where we can specify ourself what differentiation we want to apply. We can both choose forward and reverse. In our experiment, we will make use of reverse because we want to see how the best differentiation compares to others.

B. Experiments

As mentioned before we will run three different types of experiments.

a) *Setup:*

First let us discuss a bit about the minor setup we do for the benchmarking tests.

We choose to opt to Euler integration, mainly because this is the simplest way to solve ODE. [11], [12]

```
const DT = 0.01
const STEPS = 400

const TRUE_α = 1.0
const TRUE_β = 0.1
...
```

Where `DT` is what we use for the Euler method, how long the distance is we travel for every step. Next, we have our amount of steps we will take before we conclude our gradients. The `TRUE_*` are the ground truth values we just assume that we have observed.

Next, because we don't have real observed data, we just generate this and state that this is our real observed data:

```
const OBS_X, OBS_Y =
generate_observations(TRUE_α, ...)
const PARAMS = [1.1, 0.09, 1.4, 0.08,
10.5, 4.8]
```

This method of experimenting is what we call synthetic experiments and will suffice for this small benchmarking.

The `PARAMS` will represent our permuted data, and is what we will use to find (eventually) our ground truth.

And at last we need to make the simulation functions for each method (some share the same function) so we can finally start experimenting. There are three versions of the same simulation for three different API's.

b) *Correctness:*

Before measuring performance, we verify that all four methods produce correct gradients. Each method is evaluated at the perturbed parameter vector and compared against a central finite difference baseline computed as:

$$\frac{\partial L}{\partial \theta_i} \approx \frac{L(\theta_i + \varepsilon) - L(\theta_i - \varepsilon)}{2\varepsilon} \tag{1}$$

with epsilon = 10^{-5}. We report both the maximum absolute error and maximum relative error across all six parameters.

TABLE I
GRADIENT CORRECTNESS ACROSS ALL FOUR AD METHODS COMPARED TO CENTRAL FINITE DIFFERENCES.

Method	Max Absolute Error	Max Relative Error
Forward (Dual)	0.161	7.24e-8
Reverse (Tape)	0.161	7.24e-8
Zygote	0.161	7.24e-8
Enzyme	0.161	7.24e-8

All four methods return identical relative errors, confirming that they agree with each other to machine precision. This validates that the timing comparisons in the following sections are between four correct implementations of the same computation.

c) *Timing:*

Next, we check how fast each method performs and how good it is for the basic Lotka-Volterra experiment.

TABLE II
GRADIENT COMPUTATION TIME FOR THE 6-PARAMETER LOTKA-VOLTERRA PROBLEM.

Method	Median	Min	Std
Forward (Dual)	15.3 µs	13.6 µs	15.9 µs
Reverse (Tape)	1.767 ms	393.4 µs	3.797 ms
Zygote	5.364 ms	1.922 ms	6.685 ms
Enzyme	5.5 µs	4.2 µs	7.36 µs

Table 2 reports the median gradient computation time for each method on the standard 6-parameter Lotka-Volterra problem, measured using BenchmarkTools over 200 runs to avoid JIT compilation noise. [13]

Enzyme is the fastest method at 5.5 µs, nearly three times faster than forward mode and roughly 1000 times faster than Zygote. This is expected, Enzyme operates at the LLVM IR level, below Julia's object system entirely, meaning intermediate values live in CPU registers rather than on the heap. No memory allocation occurs during the backward pass. [8], [10]

Forward mode at 15.3 µs is the second fastest despite being a simpler implementation. For 6 parameters it runs 6 separate simulation passes, but each pass is lightweight, it carries only two floating point values per operation and allocates nothing extra beyond the Dual numbers themselves.

The manual reverse mode tape is significantly slower at 1.767 ms. While it correctly computes all gradients in a single backward pass, the tape infrastructure introduces overhead: every operation during the forward pass allocates a `TapeEntry` struct onto the heap, and the backward pass must walk through thousands of these entries.

Zygote is the slowest at 5.364 ms, despite also being reverse mode. As discussed by the Julia community [14], Zygote generates the backward pass as a chain of closures, each of which is a heap allocation. For a simulation with 400 timesteps and roughly 10 scalar operations per step, this produces thousands of heap objects per gradient call, confirmed by the high standard deviation of 6.685 ms indicating frequent garbage collector interference. This behaviour is well-suited to machine learning workloads dominated by large matrix operations, but is a poor fit for tight scalar simulation loops.

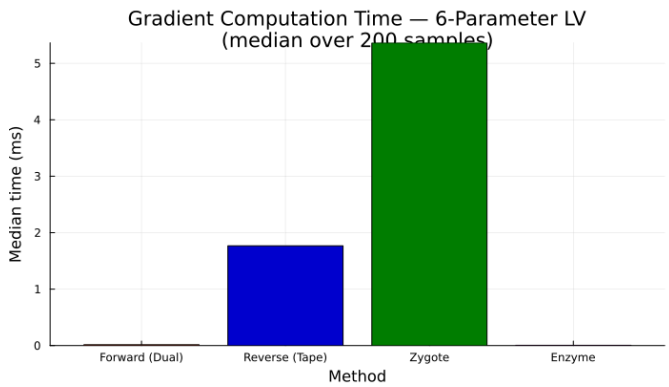

Fig. 3. Bar diagram timing

d) *Scaling:*

To observe how each method behaves as the problem grows, we extend the Lotka-Volterra system to n interacting species arranged in a ring, giving 2n parameters total, n growth rates and n interaction coefficients. We sweep from n=4 to n=24 parameters and measure the median gradient computation time over 50 samples at each size.

TABLE III
MEDIAN GRADIENT COMPUTATION TIME AS THE NUMBER OF PARAMETERS INCREASES.

Parameters	Forward	Reverse	Zygote	Enzyme
4	1.07 ms	2.14 ms	79.02 ms	0.10 ms
8	5.36 ms	3.80 ms	140.18 ms	0.20 ms
12	13.29 ms	4.32 ms	219.33 ms	0.23 ms
16	18.12 ms	7.15 ms	264.94 ms	0.27 ms
20	29.51 ms	10.02 ms	324.76 ms	0.23 ms
24	36.01 ms	11.04 ms	405.70 ms	0.40 ms

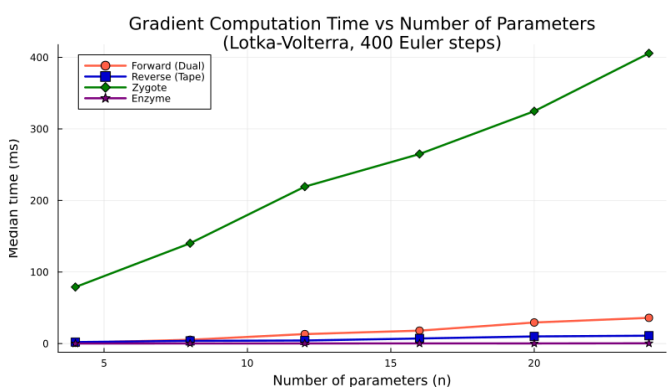

Fig. 4. Scaling graph plot

We can see that Enzyme stays stable, and this was to be expected because everything is done in LLVM, so values stay in CPU registers.

Next, we have the tape simulation. This is broadly O(1) with respect to the number of parameters as expected, though the tape overhead grows slightly because a larger system means more operations recorded per step. Notably at n=4 it is actually slower than forward mode, for very small parameter counts the tape allocation and backward pass overhead outweighs the cost of simply running a few extra forward passes. The crossover point where reverse mode becomes faster than forward mode occurs around n=8 parameters.

Forward mode grows roughly linearly. This is consistent with its O(n) cost: each additional parameter requires one full additional simulation pass.

Zygote scales linearly like forward mode but starts at a much higher baseline, because its per-operation heap allocation cost dominates regardless of parameter count.

e) *Code complexity:*

Beyond performance, the four methods differ significantly in how much code they require the programmer to write.

Forward mode requires no changes to the simulation function at all. The plain Julia code runs unchanged because operator overloading on the Dual struct propagates derivatives automatically through every arithmetic operation. Computing the gradient is a single call:

```
dual_gradient(p) do args...
    simulate_lv_scalar(args...)
end
```

Zygote is equally simple from the user's perspective, the same unmodified simulation function is passed directly to Zygote.gradient. The entire gradient computation is one line:

```
Zygote.gradient(simulate_lv_scalar, p...)
```

The simplicity of both forward mode and Zygote comes from the same underlying reason: the simulation is written in plain Julia and the AD system handles differentiation invisibly.

Enzyme requires a small but explicit change. The simulation must be rewritten to accept a `Vector{Float64}` instead of individual scalar arguments, and the caller must manually allocate a shadow array and annotate each argument with `Active`, `Const`, or `Duplicated`:

```
d = zeros(Float64, length(p))
Enzyme.autodiff(ReverseWithPrimal,
simulate_lv_vector, Active, Duplicated(p,
d))
```

Manual reverse mode is by far the most complex. The simulation cannot be written in plain Julia, every single arithmetic operation must be manually routed through the tape, replacing natural expressions like $\alpha * x - \beta * x * y$ with explicit tape calls:

```
s_αx   = taped_mul(tape, s_α, s_x)
s_βxy  = taped_mul(tape, taped_mul(tape,
s_β, s_x), s_y)
s_dx   = taped_sub(tape, s_αx, s_βxy)
```

What is two lines of readable simulation code becomes ten lines of tape manipulation. This verbosity scales with every operation in the simulation, making manual reverse mode impractical for larger problems despite its correctness.

IV. Conclusion

So overall Enzyme comes on top. It performs the best and scales the best with many inputs. Besides that it is also fairly easy to write.

If you want more inside on how your code works, you could write it yourself. Depending on what the purpose is, you might choose forward mode over reverse mode. Forward mode is especially easy to implement and fairly fast for small amount of inputs. But it scales fairly poorly.

References

[1] Wikipedia contributors, "Automatic Differentiation." Wikipedia, The Free Encyclopedia, 2025.

[2] J. Shi, "Automatic Differentiation: Forward and Reverse." 2022.

[3] Wikipedia contributors, "Lotka–Volterra equations." Wikipedia, The Free Encyclopedia, 2025.

[4] J. Revels, M. Lubin, and T. Papamarkou, "Forward-Mode Automatic Differentiation in Julia," 2016, [Online]. Available: https://arxiv.org/abs/1607.07892

[5] JuliaDiff Contributors, "ForwardDiff.jl: Forward Mode Automatic Differentiation for Julia." [Online]. Available: https://github.com/JuliaDiff/ForwardDiff.jl

[6] JuliaDiff Contributors, "ReverseDiff.jl: Reverse Mode Automatic Differentiation for Julia." [Online]. Available: https://github.com/JuliaDiff/ReverseDiff.jl

[7] ApX Machine Learning, "Automatic Differentiation with Zygote.jl." 2026.

[8] The Zygote.jl Developers, "Zygote.jl Documentation, Version 0.5." 2020.

[9] The Enzyme.jl Developers, "Enzyme.jl Documentation (Stable)." 2026.

[10] DeepWiki, "Enzyme.jl – EnzymeAD/Enzyme.jl Overview." 2025.

[11] Wikipedia contributors, "Euler Method." Wikipedia, The Free Encyclopedia, 2025.

[12] Petersd, "Euler's Method – Demo." 2025.

[13] JuliaCI, "BenchmarkTools.jl Documentation (Stable)." 2026.

[14] cortner, "Zygote Performance." 2019.